

SMART MULTI-MODAL ASSISTIVE OBJECT LOCATOR SYSTEM FOR VISUALLY IMPAIRED USERS USING ESP32, RASPBERRY PI, AND HUSKYLENS

MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

SECTION 1

GROUP 6

MATRIC NUMBER	NAME
2312479	MUHAMMAD HAZIMUDDIN BIN FAIRUZ
2312369	MUHAMMAD FAHIM BIN AHMAD NORISHAM
2312451	MUHAMMAD ZAMARUL AZIM BIN ZULKHIBRI

DATE OF SUBMISSION

Wednesday, 24th December 2025

Abstract

Visually impaired people often have difficulty finding personal items such as wallets, keys, or mobile phones inside their homes. Most existing item-tracking products depend on smartphone screens and visual maps, which makes them hard for blind users to use. This project presents an indoor Smart Assistive Item Locator System that helps visually impaired users find misplaced objects using simple and intuitive feedback methods.

The proposed system employs a centralized indoor architecture consisting of two ESP32 microcontrollers and a Raspberry Pi acting as a home-based control unit. A wearable bracelet equipped with an ESP32 allows user interaction via push buttons and delivers guidance using a vibration motor. An object-mounted ESP32 activates an acoustic buzzer to indicate the location of the selected item. Communication between system components is achieved through WiFi within the home network, enabling reliable indoor operation and centralized decision-making.

To enhance object confirmation and directional guidance, a HuskyLens vision sensor is integrated into the wearable unit. When the target object is visually detected, tactile feedback is provided through vibration, allowing the user to orient themselves toward the object without relying solely on sound. A single 3.7 V Li-ion battery powers the wearable system, with a TP4056 charging module and a DC-DC boost converter ensuring stable operation.

The proposed system emphasizes affordability, accessibility, and user-centered design, offering an effective indoor assistive solution that improves independence and usability for visually impaired individuals.

Keywords: Visually Impaired Assistance Indoor Item Locator Assistive Technology
ESP32 Raspberry Pi HuskyLens Tactile Feedback Smart Wearable System

Table of Contents

Abstract.....	1
1. Introduction.....	3
2. Methodology.....	5
2.1 Project Development.....	5
2.2 Schematic Diagram and Prototype Setup.....	7
3. References.....	9
Certificate of Originality and Authenticity.....	10

1. Introduction

Visually impaired individuals face continuous challenges in performing daily activities independently, particularly when locating personal belongings within indoor environments. Common items such as wallets, keys, and mobile phones are often misplaced, and without visual cues, identifying their location can be time-consuming and mentally exhausting. This limitation not only reduces efficiency but also affects the user's confidence and independence in managing daily routines.

Existing item locator solutions, such as Bluetooth trackers and smart tags, are widely available in the consumer market. However, most of these systems rely heavily on smartphone-based visual interfaces, maps, or screen notifications, making them unsuitable for blind or visually impaired users. While some devices provide audible alerts, they lack directional guidance, tactile feedback, and intelligent confirmation of the target object. As a result, visually impaired users may still struggle to accurately locate and physically retrieve the desired item, especially in cluttered indoor environments.

Recent advancements in embedded systems, wireless communication, and artificial intelligence have enabled the development of assistive technologies that are more accessible and user-centered. Microcontrollers such as the ESP32 offer low-power wireless communication and real-time input-output control, while microprocessors like the Raspberry Pi provide centralized processing and system coordination. In addition, AI-based vision sensors such as HuskyLens enable real-time object recognition without requiring complex external computation or visual displays.

This project proposes a Smart Multi-Modal Assistive Object Locator System designed specifically for visually impaired users in indoor environments. The system integrates auditory, tactile, and visual intelligence to assist users in locating misplaced personal items effectively. A wearable bracelet equipped with an ESP32 microcontroller serves as the primary user interface, allowing users to initiate item searches using simple push buttons. Target objects such as wallets, keys, or mobile phones are equipped with ESP32-based locator units that activate audible buzzers upon receiving search commands.

A Raspberry Pi functions as a centralized home control unit, coordinating communication between the wearable bracelet and the object locator units through WiFi. This centralized architecture enhances system reliability and simplifies indoor operation by enabling

structured communication and decision-making. To further improve accuracy and usability, a HuskyLens vision sensor is integrated into the wearable bracelet. Once the user is near the sound source, the HuskyLens detects and recognizes the target object, providing tactile feedback through a vibration motor to guide the user toward the precise location of the item.

Compared to existing item locator systems, the proposed solution emphasizes accessibility, multimodal feedback, and independence from visual interfaces. By combining audio alerts, tactile vibration, and AI-based object recognition within a wearable system, this project aims to improve the independence, efficiency, and overall quality of life for visually impaired individuals. From a Mechatronics System Integration perspective, the project demonstrates effective integration of microcontrollers, microprocessors, wireless communication, and intelligent sensing within a unified assistive system.

Table 1 Shows the Comparison of Existing Item Locator Systems

Ref / Year	Title	Advantages	Disadvantages	Compared to Proposed System
9/2021	Apple AirTag	High location accuracy, wide ecosystem support	Visual-only interface, no tactile guidance, requires smartphone	Proposed system provides vibration and audio guidance without visual dependency
2022	Tile Bluetooth Tracker	Low cost, audible alert	Limited range, smartphone-dependent, no intelligent guidance	Proposed system offers centralized control and vision-assisted confirmation
3/2019	Bluetooth-Based Indoor Object Finder	Simple implementation, buzzer alert	No directional or tactile feedback, not designed for blind users	Proposed system integrates tactile vibration and AI-based object detection
Proposed	Smart Assistive Item Locator System	Multimodal feedback, vision-assisted guidance, affordable, accessible	Limited to indoor environments	Designed specifically for visually impaired users

2. Methodology

2.1 Project Development

The overall system architecture is illustrated using a block diagram that represents the interaction between input devices, processing units, communication modules, and output mechanisms. The proposed project consists of three main subsystems: the wearable bracelet unit, the object locator unit, and the home central control unit.

System 1: Wearable Bracelet Unit

Based on the illustration in Fig. 1, it consists of an ESP32 microcontroller connected to three push buttons, a vibration motor, and the HuskyLens module. Each button corresponds to a specific object (wallet, keys, or phone). Upon button activation, a request is sent to the Raspberry Pi, which coordinates the object search process. This subsystem is built around an ESP32 microcontroller and serves as the primary user interface. It includes three push buttons corresponding to different personal items, a vibration motor for tactile feedback, and a HuskyLens vision sensor for object detection. User input is captured through button presses, and guidance is provided through vibration when the target object is detected.

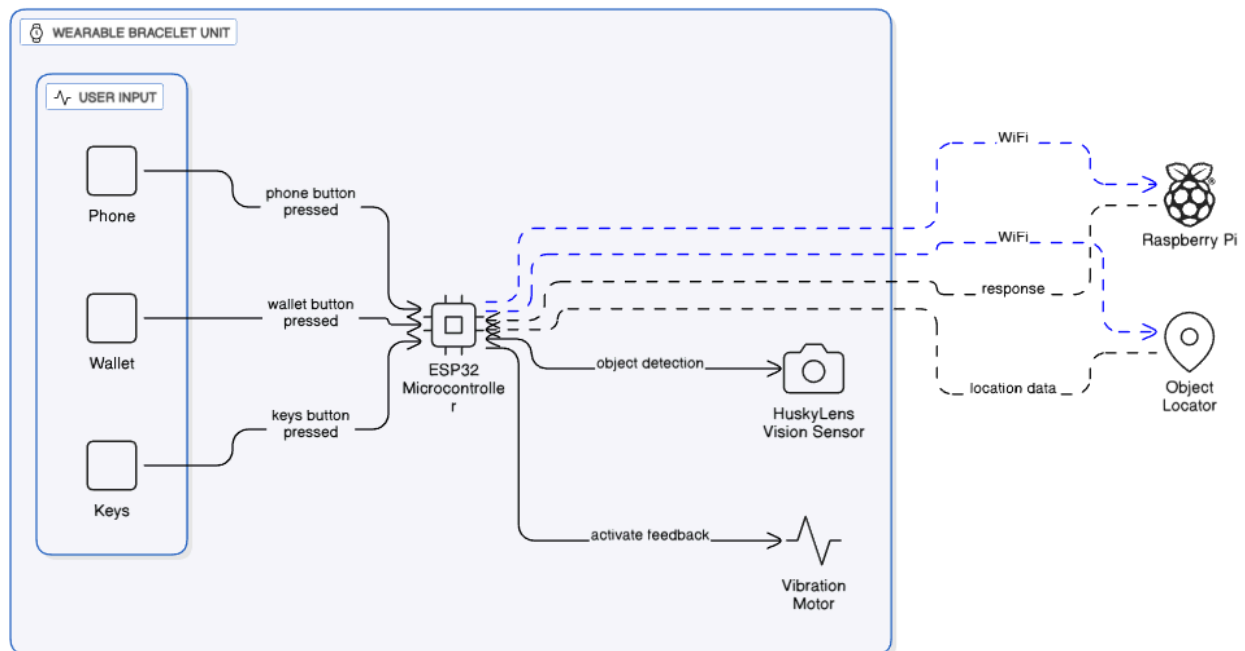


Figure 1.0 Shows the flowchart of the System 1: Wearable Bracelet Unit

System 2: Object Locator Unit

The object locator unit consists of an ESP32 microcontroller attached to the target item. Three buzzers are connected to the microcontroller to provide audible cues indicating the object's location. Upon receiving commands from the Raspberry Pi, the appropriate buzzer is activated. As shown in Fig. 2 below, represents the object locator unit attached to the target item. This subsystem uses an ESP32 microcontroller connected to three buzzers, each representing a distinct object. Upon receiving a command from the Raspberry Pi, the corresponding buzzer is activated to provide an audible cue indicating the object's location.

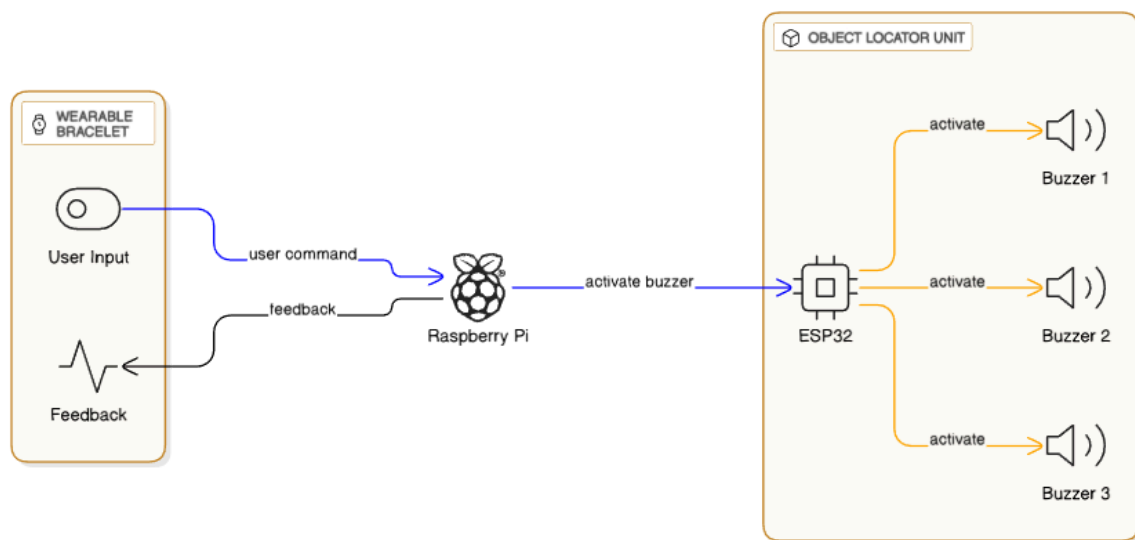


Figure 2.0 Shows the flowchart of the System 2: Object Locator Unit

System 1 and System 2: Home Central Control Unit

The Raspberry Pi functions as the central control unit and coordinates communication between the wearable bracelet and the object locator units via WiFi. It manages system logic, processes search requests, and determines when to activate audio or tactile feedback. The centralized architecture ensures reliable indoor operation and simplifies system management.

A flowchart illustrates the operational sequence of the system, beginning with user input, followed by communication with the Raspberry Pi, object detection, and feedback activation. It comprises the Raspberry Pi, which functions as the central control unit. It manages wireless communication, processes object detection status, and determines when to activate vibration or buzzer feedback. The flowchart presented in Fig. 4 illustrates the operational logic of the entire system, from button press detection to object localization and guidance feedback.

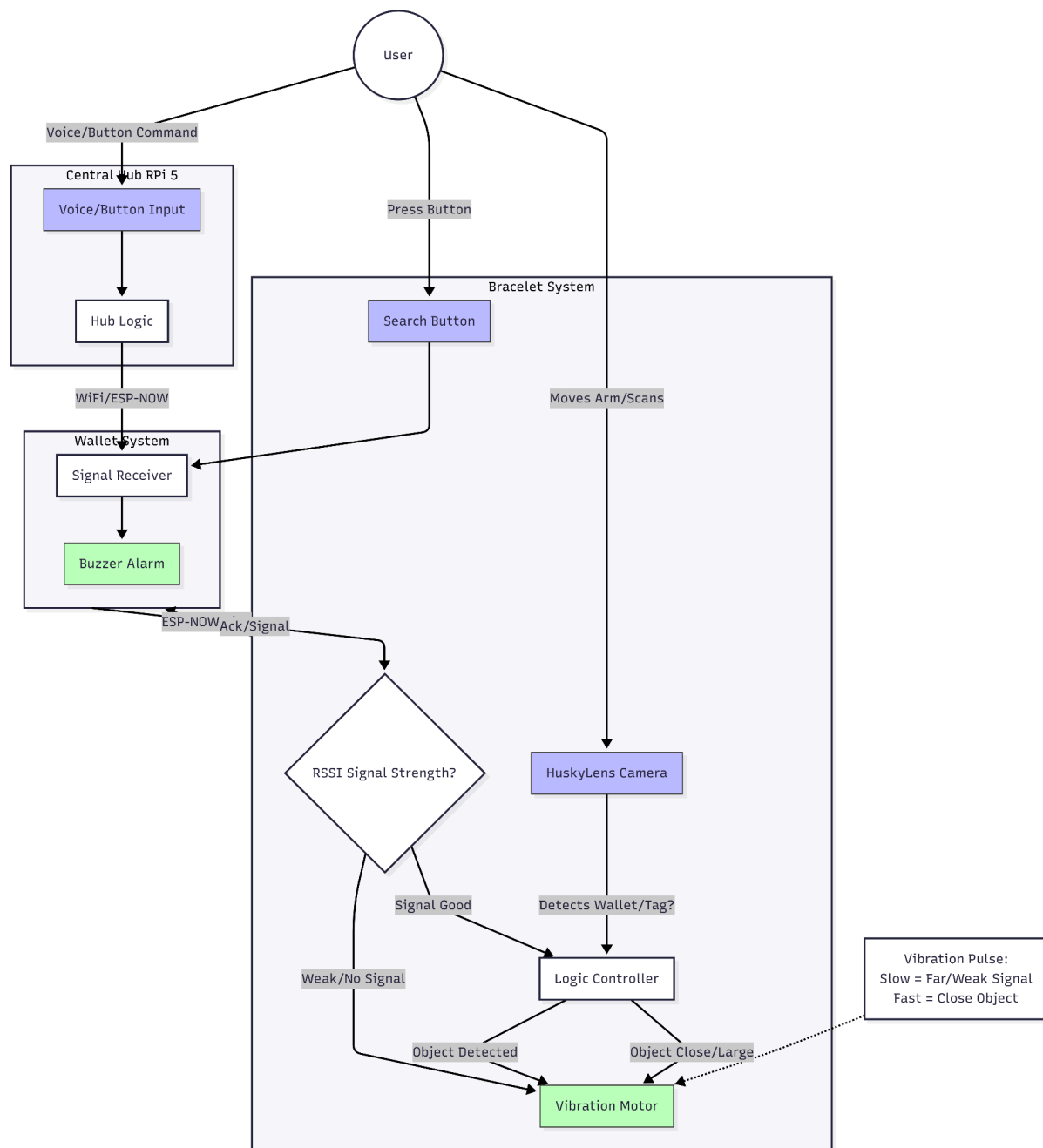


Figure 3.0 Shows the flowchart of the Object Locator Unit

Based on **Figure 3.0** illustrates the complete operational logic of the Assistive Blind Object Locator System. The architecture is divided into three primary subsystems: the Bracelet System (Wearable Master), the Wallet System (Target Device), and the Central Hub (Stationary Backup). These systems interact through wireless protocols (ESP-NOW/Wi-Fi) to provide redundancy and multi-sensory feedback to the user.

The system workflow is categorized into three distinct functional cases as detailed below:

2.1.1 Active Audio Search (Remote Location)

This workflow allows the user to locate the wallet audibly when it is out of immediate reach but within the house. As depicted in the "User" interaction paths (Top Center), there are two methods to trigger this alarm:

- **Primary Method (Bracelet):** The user presses the physical Search Button on the Bracelet System. This sends a direct signal to the Signal Receiver on the Wallet System, instantly triggering the Buzzer Alarm.
- **Secondary Method (Central Hub Backup):** If the bracelet is disconnected or unavailable, the user can issue a Voice/Button Command to the Central Hub RPi 5. The Hub Logic processes this request and transmits a signal via Wi-Fi/ESP-NOW to the Wallet System, activating the same Buzzer Alarm.

2.1.2 Passive Tethering (Anti-Loss Monitoring)

To prevent the user from losing the wallet initially, the system runs a continuous background monitoring loop shown in the lower-left logic path:

- **Signal Acquisition:** The Bracelet System continuously exchanges "Ack/Signal" packets with the Wallet System using the low-latency ESP-NOW protocol.
- **RSSI Evaluation:** The Logic Controller analyzes the RSSI Signal Strength (Received Signal Strength Indicator) of these packets.
- **Loss Alert:** If the signal is determined to be "Weak/No Signal" (indicating the user has walked away from the wallet), the logic triggers the Vibration Motor on the bracelet. This acts as a preventative "virtual leash," alerting the user immediately if they leave the wallet behind.

2.1.3 Visual Haptic Guidance (Precision Pinpointing)

Once the user is in close proximity to the wallet (guided by the buzzer or memory), the system switches to precision guidance to help the user physically grasp the item:

- **Visual Recognition:** As the user Moves Arm/Scans the area, the HuskyLens Camera integrated into the bracelet processes the video feed in real-time.

- **Object Detection Logic:** The camera queries "Detects Wallet/Tag?". Upon positive identification of the wallet (or its specific marker), the Logic Controller engages the Vibration Motor.
- **Variable Feedback Loop:** As noted in the diagram's legend (*Vibration Pulse*), the system utilizes a variable haptic language:
 - **Slow Pulse:** Indicates the object is detected but is distant or small in the frame.
 - **Fast Pulse:** Indicates the object is close or large in the frame, signaling the user to reach out.

This architecture ensures high reliability by combining **redundancy** (RPi Hub acts as a fail-safe for the Bracelet), **prevention** (RSSI Tethering prevents loss before it happens), and **accessibility** (translating visual data from the Camera into haptic vibrations), thereby creating a complete safety net for the visually impaired user.

2.2 Schematic Diagram and Prototype Setup

The schematic diagram shows how all the parts are connected in each part of the system. The ESP32 in the wearable bracelet is connected to three push buttons, each with a pull-down resistor to make sure the button signals are stable. A small circuit with a transistor is used to turn the vibration motor on and off without drawing too much current from the ESP32. A diode is placed across the motor to protect the circuit from sudden voltage spikes.

The ESP32 on the object controls three buzzers, each connected to its own pin through a resistor to limit the current. The ESP32 modules and the Raspberry Pi communicate wirelessly through WiFi, so there is no need for wires between the parts.

The wearable unit is powered by a single 3.7 V Li-ion battery. A TP4056 charging module is used to charge the battery and provide power to the ESP32, vibration motor, and a DC-DC boost converter. The boost converter gives a steady 5 V needed for the HuskyLens. The Raspberry Pi uses a 5 V power supply and stays on all the time as the home hub.

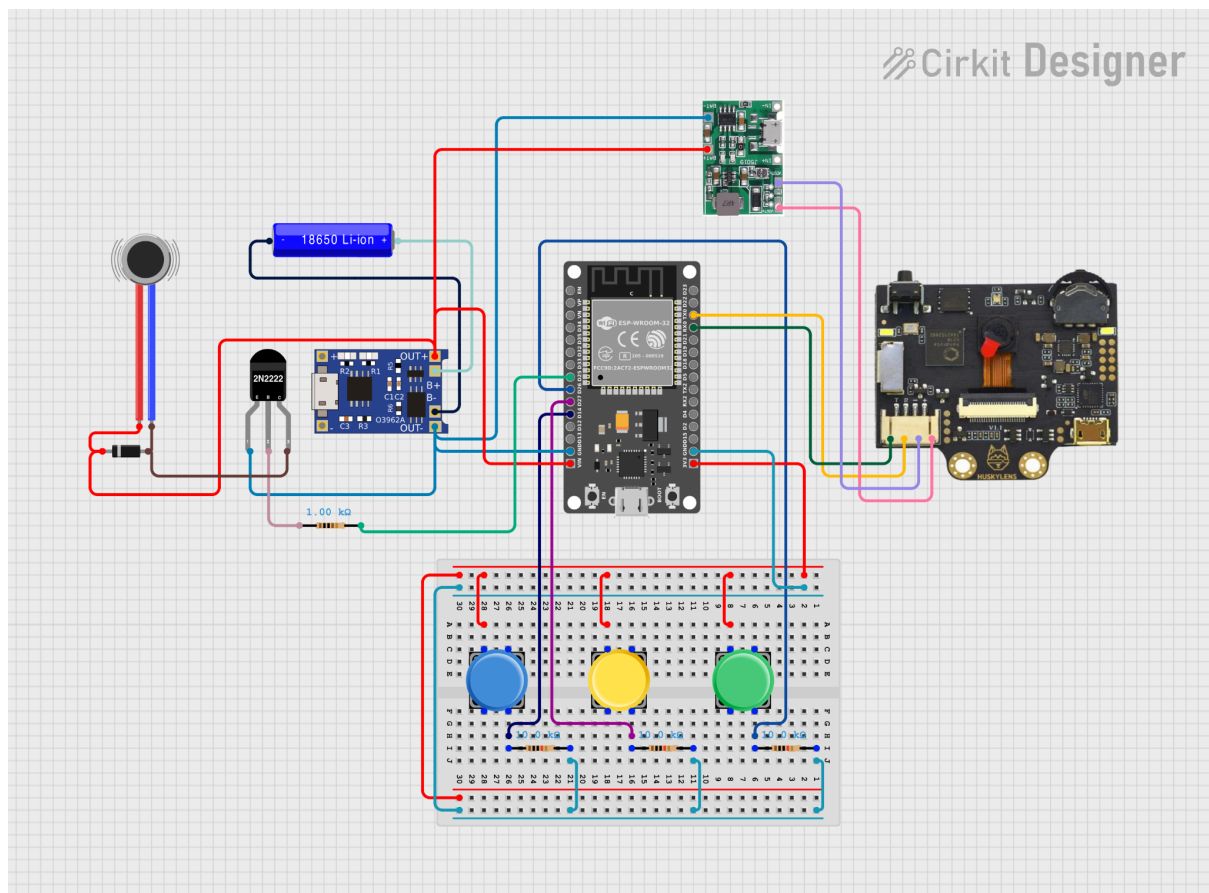


Figure 2.2.0 Electrical design for ESP32-Bracelet

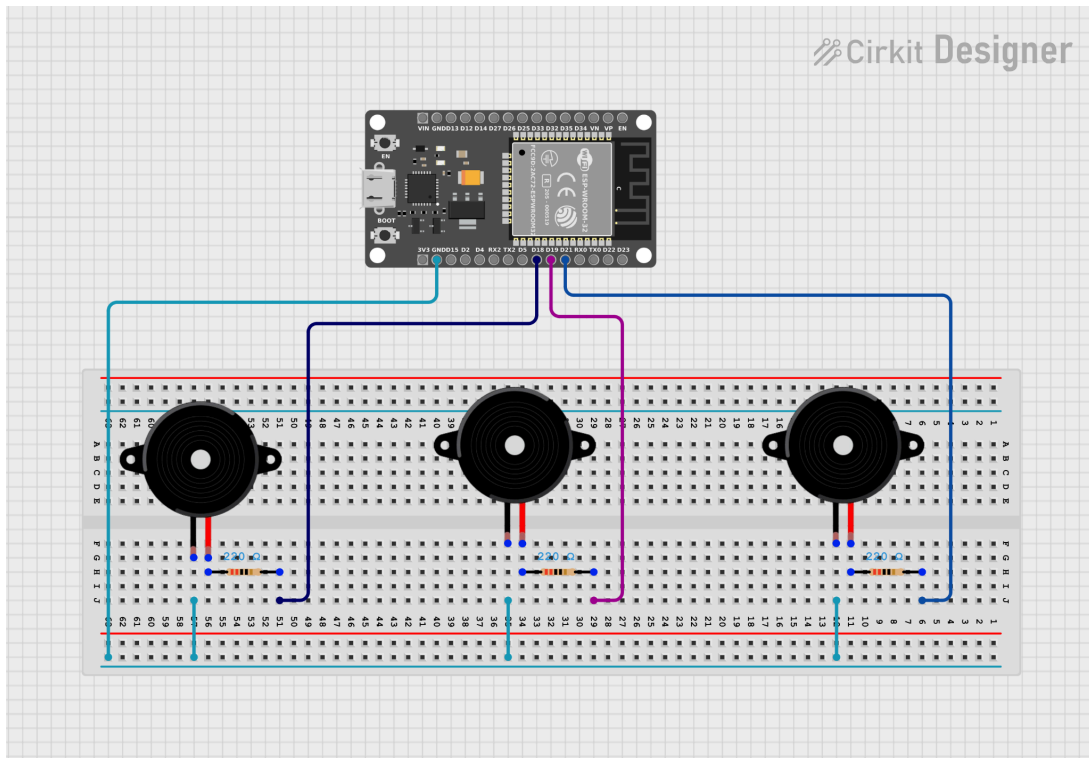


Figure 2.2.1 Electrical design for ESP32-Object Unit

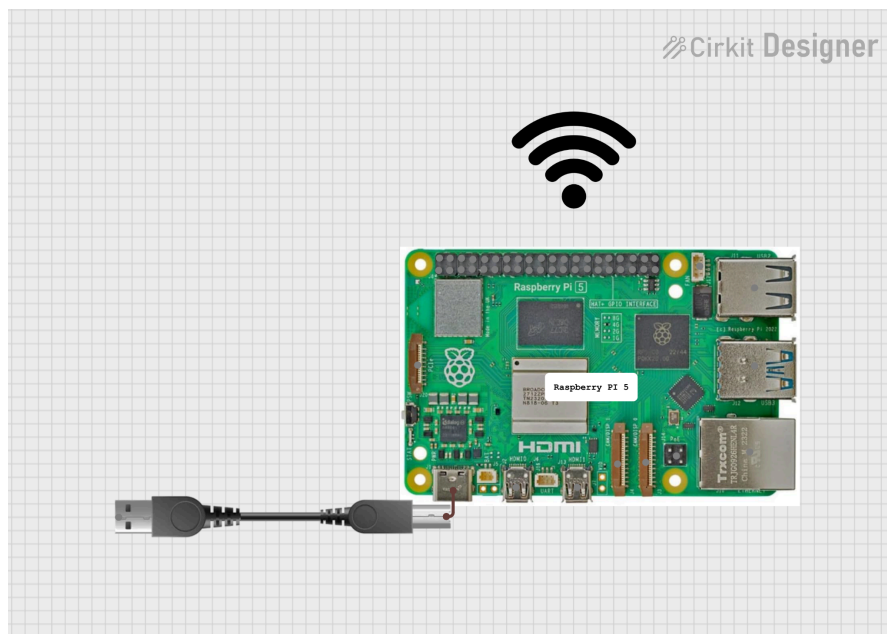


Figure 2.2.2 Smart Multi-Modal Assistive Object Locator System

2.3 Materials and Components

2.3.1 ESP32-Bracelet

ESP32 Development Board	1
Buttons	3
Resistor 220Ω	1
Transistor (2N222)	1
DC-DC Boost Converter	1
Charging Module (TP4056)	1
Lithium Ion Battery	1
Diode (1N4148)	1
Breadboard	1
Jumper Wires	

2.3.2 ESP32-Object Unit

ESP32 Development Board	1
Resistor 220Ω	1
Buzzer	3
Breadboard	
Jumper Wires	

2.4 Coding Snippet

2.4.1 Coding for ESP32-Bracelet

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <Wire.h>
#include "HUSKYLENS.h"

// --- CONFIGURATION ---
const char* ssid = "Jimmy";
const char* password = "jemmeh123";
const char* walletIP = "10.154.161.164"; // <--- CHANGE THIS to your Wallet
ESP32 IP
const int buttonPin = 25;                // Updated to Pin 25

HUSKYLENS huskylens;
bool huskyConnected = false;

// Logic Variables
bool buttonActive = false;               // Is the button toggle ON?
bool lastButtonState = HIGH;             // For edge detection
unsigned long lastDebounce = 0;

bool huskyActive = false;                // Is the camera currently seeing the
object?
bool lastHuskySent = false;              // To prevent spamming WiFi

void setup() {
    Serial.begin(115200);
    pinMode(buttonPin, INPUT_PULLUP);
    Wire.begin(21, 22); // SDA, SCL

    // 1. Connect WiFi
    WiFi.begin(ssid, password);
    Serial.print("Connecting to WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi connected");

    // 2. Init HuskyLens (NON-BLOCKING)
    // We use an 'if' instead of 'while' so the button works even if camera
    fails
    if (huskylens.begin(Wire)) {
        Serial.println("HuskyLens Online!");
        huskyConnected = true;
    } else {
        Serial.println("WARNING: HuskyLens not found (Check wiring/I2C
Protocol).");
        Serial.println("Button will still work.");
        huskyConnected = false;
    }
}
```

```

void loop() {
    // --- PART A: BUTTON TOGGLE LOGIC ---
    int reading = digitalRead(buttonPin);

    // Check if button changed (Pressed LOW)
    if (reading == LOW && lastButtonState == HIGH && (millis() - lastDebounce
    > 200)) {
        buttonActive = !buttonActive; // Toggle the state (On becomes Off, Off
    becomes On)
        lastDebounce = millis();

        Serial.print("Button Toggled: ");
        Serial.println(buttonActive ? "ON" : "OFF");

        // Send command immediately
        if (buttonActive) sendRequest("/button/on");
        else sendRequest("/button/off");
    }
    lastButtonState = reading;

    // --- PART B: HUSKYLENS LOGIC ---
    bool currentDetecting = false;

    if (huskyConnected && huskylens.request()) {
        if (huskylens.isLearned() && huskylens.available()) {
            currentDetecting = true;
        }
    }

    // Only send WiFi request if the camera status CHANGES (to save speed)
    if (currentDetecting != huskyActive) {
        huskyActive = currentDetecting;
        if (huskyActive) {
            Serial.println("Object Found -> Sending Husky ON");
            sendRequest("/husky/on");
        } else {
            Serial.println("Object Lost -> Sending Husky OFF");
            sendRequest("/husky/off");
        }
    }

    delay(50); // Small delay for stability
}

void sendRequest(const char* path) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        String url = "http://" + String(walletIP) + path;
        http.begin(url);
        int httpCode = http.GET();
        http.end();
    } else {
        Serial.println("WiFi Disconnected!");
    }
}

```


2.4.2 Coding for ESP32-Object Unit

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "Jimmy";
const char* password = "jemmeh123";

// --- PIN DEFINITIONS ---
const int buzzerPin = 5; // Buzzer Pin
const int ledPin = 18;   // LED Pin (Only for HuskyLens)

WebServer server(80);

// State variables to track who wants the buzzer ON
bool buttonRequest = false;
bool huskyRequest = false;

// --- LOGIC: Update Outputs based on Requests ---
void updateOutputs() {
    // 1. BUZZER LOGIC:
    // Ring if Button is pressed OR Husky sees object
    if (buttonRequest || huskyRequest) {
        digitalWrite(buzzerPin, HIGH);
    } else {
        digitalWrite(buzzerPin, LOW);
    }

    // 2. LED LOGIC:
    // Light up ONLY if Husky sees object
    if (huskyRequest) {
        digitalWrite(ledPin, HIGH);
    } else {
        digitalWrite(ledPin, LOW);
    }
}

// --- WEB HANDLERS ---
void handleButtonOn() {
    buttonRequest = true;
    updateOutputs();
    server.send(200, "text/plain", "Button ON");
    Serial.println("Command: Button ON (Buzzer Only)");
}

void handleButtonOff() {
    buttonRequest = false;
    updateOutputs();
    server.send(200, "text/plain", "Button OFF");
    Serial.println("Command: Button OFF");
}

void handleHuskyOn() {
    huskyRequest = true;
    updateOutputs();
    server.send(200, "text/plain", "Husky ON");
    Serial.println("Command: Husky ON (Buzzer + LED)");
}
```

```
void handleHuskyOff() {
    huskyRequest = false;
    updateOutputs();
    server.send(200, "text/plain", "Husky OFF");
    Serial.println("Command: Husky OFF");
}

void setup() {
    Serial.begin(115200);

    // Setup Pins
    pinMode(buzzerPin, OUTPUT);
    pinMode(ledPin, OUTPUT);

    // Start OFF
    digitalWrite(buzzerPin, LOW);
    digitalWrite(ledPin, LOW);

    // Connect WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nReceiver Ready!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());

    // Register URLs
    server.on("/button/on", handleButtonOn);
    server.on("/button/off", handleButtonOff);
    server.on("/husky/on", handleHuskyOn);
    server.on("/husky/off", handleHuskyOff);

    server.begin();
}

void loop() {
    server.handleClient();
}
```

3. References

B S, N., R, P., Haumsabhavi, S. S., A, S. S., & T P, R. (2025). Smart Indoor Navigation for the Blind Using Li-Fi and Voice Assistance. *IJARCCCE*, 14(12).
<https://doi.org/10.17148/ijarcce.2025.141235>

Life360. (2025). *Track Your Family, Friends, Keys, Wallets, and More | Life360 + Tile*.
Life360 | Family Tracking App | Location Sharing & Family Safety.
<https://intl.life360.com/tile-trackers>

Pratap, A., Kumar, S., & Chakravarty, S. (2025). *Adaptive Object Detection for Indoor Navigation Assistance: A Performance Evaluation of Real-Time Algorithms*.
ArXiv.org. <https://arxiv.org/abs/2501.18444>

Robo Tech Valley. (2025). *HuskyLens – AI-Powered Vision Sensor for Robotics & DIY Projects - Store - Robo Tech Valley*. Store - Robo Tech Valley.
<https://store.robotechvalley.com/product/huskylens-ai-powered-vision-sensor-for-robotics-diy-projects/>

Sethuraman, S. C., Tadkapally, Gaurav R, Mohanty, S. P., Galada, G., & Subramanian, A. (2023). *MagicEye: An Intelligent Wearable Towards Independent Living of Visually Impaired*. ArXiv.org. <https://arxiv.org/abs/2303.13863>

u-blox. (2020, December 15). *Bluetooth Indoor Positioning*. U-Blox.
<https://www.u-blox.com/en/technologies/bluetooth-indoor-positioning>

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature: 

Name: Muhammad Hazimuddin Bin Fairuz

Matric Number: 2312479

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Zamarul Azim Bin Zulkhibri

Matric Number: 2312451

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Fahim Bin Ahmad Norisham

Name: Matric Number: 2312369

Read /

Understand /

Agree /